

Devoir surveillé

Durée 2h

Nom :

Prénom :

Num. inscription :

Aucun document, outil de calcul ou de communication autorisé.

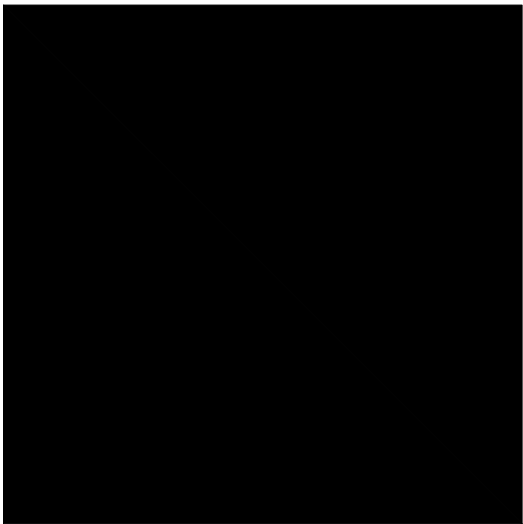
Exercice 1 (QCM - 5 pts)

Vous obtiendrez 0,5 point aux questions si vous cochez exactement toutes les bonnes réponses. Une mauvaise réponse cochée entraîne des points négatifs.

- Avec un *i-node*, sous *ext2*...
 - ☐ ...les blocs de donnée d'un fichier sont stockés de manière contigüe sur le disque
 - ☐ ...les blocs de donnée d'un fichier peuvent être stockés à différents endroits sur le disque
 - ☐ ...la taille maximale d'un fichier dépend de la taille des blocs du disque
 - ☐ ...un bloc de données ne peut pas contenir les données de plusieurs fichiers
- Quelle(s) affirmation(s) est/sont juste(s) ?
 - ☐ La taille d'un fichier peut être inférieure à la taille occupée sur le disque
 - ☐ La taille d'un fichier est identique à la taille occupée sur le disque
 - ☐ La taille d'un fichier peut être supérieure à la taille occupée sur le disque
 - ☐ La taille occupée sur le disque dépend de la taille des blocs
- Concernant les structures en C :
 - ☐ Elles sont alignées automatiquement par le compilateur
 - ☐ Leur taille dépend de l'ordre de définition des champs
 - ☐ Il est plus efficace de réaliser des allocations dynamiques pour les utiliser
 - ☐ Pour une question de portabilité, il faut éviter de les utiliser
- Sur une architecture 64 bits, quelle est la taille de la structure suivante en utilisant `gcc` sous `Linux` ?

```
typedef struct {  
    long a;  
    char b;  
    char c;  
    int d;  
    short e;  
} structure_t;
```

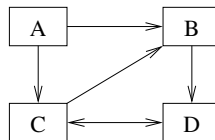
- ☐ 16o ☐ 20o ☐ 24o ☐ 32o
- En considérant la question 4, combien d'octets de bourrage sont ajoutés par le compilateur ?
 - ☐ 0 ☐ 4 ☐ 8 ☐ Une autre réponse
 - En considérant la question 4, quelle est la taille minimale que l'on peut obtenir en réarrangeant les champs de la structure (sans modifier les types) ?
 - ☐ 8 ☐ 12 ☐ 18 ☐ Une autre réponse



7. Les tubes anonymes :

- ☐ Ils ne permettent pas l'écriture simultanée de plusieurs processus
- ☐ L'écriture y est atomique
- ☐ La création d'un tube anonyme entraîne l'ajout d'une entrée dans le système de fichiers
- ☐ La création nécessite deux descripteurs de fichier

8. Combien de sémaphores de *Dijkstra* sont nécessaires au minimum pour résoudre les contraintes du diagramme de précedence suivant (A, B, C et D étant des sections de code de processus quelconques) ?

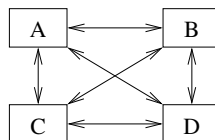


- ☐ 1 ☐ 2 ☐ 3 ☐ 4

9. Soit le diagramme de précedence de la question 8. Combien d'opérations (hors `init`) sont nécessaires ?

- ☐ 4 ☐ 6 ☐ 8 ☐ 10

10. Combien de sémaphores de *Dijkstra* sont nécessaires au minimum pour résoudre les contraintes du diagramme de précedence suivant (A, B, C et D étant des sections de code de processus quelconques) ?



- ☐ 1 ☐ 3 ☐ 6 ☐ 12

Annexes : tailles et alignements

Nous considérons les tailles et les alignements suivants (correspondant à *gcc* sous *Linux*, sur une architecture 64 bits) :

Type	Taille	Aligne.	Type	Taille	Aligne.	Type	Taille	Aligne.
char	1o	1o	short	2o	2o	int	4o	4o
long	8o	8o	float	4o	4o	double	8o	8o
long long	8o	8o	long double	16o	16o	pointer	8o	8o

*Sauf indications contraires, la gestion d'erreur **ne doit pas** être réalisée et les inclusions ne doivent pas être spécifiées. Tout réponse doit être justifiée.*

Exercice 2 (L'hydre de Lerne à n têtes - 7 pts)

L'hydre de Lerne est un animal mythologique constitué de plusieurs têtes : lorsque l'une d'elles est coupée, elle repousse. Par contre, si toutes les têtes sont coupées simultanément, l'hydre meurt (en tout cas, nous utiliserons cette interprétation pour cet exercice).

Nous souhaitons simuler le comportement de l'hydre à l'aide d'un processus et de n processus fils (qui correspondent aux têtes). Pour trancher une tête, nous envoyons un signal `SIGUSR1` à l'un des fils. Le père crée un nouveau fils lorsqu'un fils meurt sauf si tous ses fils sont morts.

1°) Quel mécanisme système permet à un processus de savoir qu'un de ses fils est terminé ? Dans un programme en C, comment un processus peut-il savoir qu'un de ses fils s'arrête ? Que doit-il faire pour libérer les ressources associées au fils ?

2°) Un fils se contente d'attendre la réception d'un signal `SIGUSR1` : à l'aide d'une alarme, il affiche régulièrement le message "*Je suis en vie (X) !*" (où X est son PID). Il affiche le message "*Je me suis fait couper (X)*" (où X est son PID) lorsqu'il reçoit le message fatidique. Donnez le code de la procédure `fils` correspondant à la routine d'un fils. Vous donnerez les fonctions/procédures supplémentaires si nécessaires.

3°) Le programme principal prend le nombre de fils en argument. Il crée les fils puis se met en attente de la mort de l'un d'eux. Pour le moment, il ne crée pas de nouvelle tête (de fils) et il s'arrête dès qu'il n'en a plus. Donnez le code du `main`.

4°) Que se passe-t-il si c'est le père qui reçoit par inadvertance le signal `SIGUSR1` ?

Pour faire repousser les têtes, nous utilisons une alarme. Le père arme une alarme de cinq secondes à la mort de l'un de ses fils. Si l'alarme arrive à expiration, le père crée les fils manquants. S'il perd un autre fils entre temps, il réarme l'alarme. Nous affichons un message "*Je vais pouvoir faire pousser mes têtes*" à la fin de l'alarme à l'aide d'un gestionnaire.

5°) Que se passe-t-il pour un appel système lorsqu'un processus reçoit un signal ? Comment s'en rendre compte dans le programme ?

6°) Expliquez comment mettre en place la gestion de l'alarme, ainsi que les modifications à apporter à la boucle principale.

Exercice 3 (Redirection - 3 pts)

Nous souhaitons réaliser un programme que nous appellerons l'*exécuteur*, qui exécute un second programme que nous appellerons le *programme client*. Le programme client lit au clavier cinq entiers puis cinq chaînes de caractères. L'exécuteur prend comme arguments le nom du programme client, ainsi que le nom d'un fichier qui contient les données à envoyer au programme client via l'entrée standard.

1°) Rappelez comment un programme peut exécuter un second programme, tout en continuant son exécution (le premier programme doit attendre la fin de l'exécution du second programme). Vous préciserez en plus, tous les appels systèmes nécessaires.

2°) Expliquez comment `dup` (ou `dup2`) peut être utilisé pour que le programme client lise les données depuis le fichier au lieu de l'entrée standard. À quel niveau, dans le programme de l'exécuteur, doit-on faire l'appel à `dup` ?

Nous souhaitons maintenant rediriger la sortie standard du programme client dans un tube anonyme. L'exécuteur récupère alors les données écrites.

3°) Expliquez ce que l'on doit mettre en place et comment. Vous donnez en plus les appels systèmes nécessaires.

Exercice 4 (Tubes - 5 pts)

Nous souhaitons faire communiquer deux processus à l'aide de deux tubes nommés.

1°) Quelle(s) est/sont la/les différence(s) entre un tube nommé et un tube anonyme ?

Le premier programme crée les deux tubes nommés puis les ouvre. Le second programme se contente de les ouvrir (en considérant que les tubes sont créés et qu'il soit donc exécuté en second). Les noms des tubes sont spécifiés en constante (`NOM_TUBE1` et `NOM_TUBE2`).

2°) Donnez la portion de code du premier programme permettant de créer les tubes nommés et de les ouvrir.

3°) Donnez la portion de code du second programme permettant d'ouvrir les tubes nommés.

Nous supposons que les programmes s'échangent des messages dont la structure est la suivante :

```
typedef struct {  
    pid_t PID;  
    char *msg;  
} message_t;
```

4°) Donnez la portion de code permettant d'initialiser puis d'écrire le contenu d'une telle structure dans un tube depuis le premier programme. Le champ `PID` contient le PID du processus courant et le champ `msg` contient une chaîne de caractères lue au clavier.

5°) Nous souhaitons que le premier programme supprime les tubes nommés dès qu'il se termine (de manière normale ou non). Expliquez comment faire et donnez les instructions à ajouter.

Annexes : quelques en-têtes de fonctions C

```

void      _exit(int code);
unsigned int alarm(unsigned int nb_secondes);
int       atexit(void (*procedure)(void));
void      clearerr(FILE *flux);
int       close(int fd);
int       closedir(DIR *repertoire);
int       creat(const char *chemin, mode_t mode);
int       dup(int ancien_fd);
int       dup2(int ancien_fd, nouveau_fd);
int       execve(const char *fichier, char *const argv[], char *const envp[]);
void      exit(int statut);
int       fcntl(int fd, int commande, ...);
int       fclose(FILE *flux);
FILE      *fdopen(int fd, const char *mode);
int       feof(FILE *flux);
int       ferror(FILE *flux);
int       fflush(FILE *flux);
int       fileno(FILE *flux);
FILE      *fopen(const char *chemin, const char *mode);
pid_t     fork();
size_t    fread(void *tampon, size_t taille, size_t nombre_elements, FILE *flux);
FILE      *freopen(const char *chemin, const char *mode, FILE *flux);
int       fseek(FILE *flux, long offset, int point_depart);
int       fstat(int fd, struct stat *tampon);
int       ftell(FILE *flux);
size_t    fwrite(const void *tampon, size_t taille, size_t nombre_elements, FILE *flux);
pid_t     getpid();
pid_t     getppid();
int       kill(pid_t pid, int numero_signal);
off_t     lseek(int fd, off_t offset, int point_depart);
int       lstat(const char *chemin, struct stat *tampon);
void      *memcpy(void *destination, const void *source, size_t taille);
void      *memset(void *tampon, int caractere, size_t nombre_elements);
int       mkfifo(const char *nomFichier, mode_t mode);
int       open(const char *chemin, int options);
int       open(const char *chemin, int options, mode_t mode);
DIR        *opendir(const char *chemin);
int       pause(void);
int       pipe(int tube[2]);
ssize_t    read(int fd, void *tampon, size_t taille);
struct dirent *readdir(DIR *repertoire);
void      rewind(FILE *flux);
void      rewinddir(DIR *repertoire);
void      (*signal(int numero_signal, void (*handler)(int)))(int)
int       sigaction(int numero_signal, const struct sigaction *action,
                    struct sigaction *ancienne_action);
int       sigaddset(sigset_t *ensemble, int numero_signal);
int       sigdelset(sigset_t *ensemble, int numero_signal);
int       sigemptyset(sigset_t *ensemble);
int       sigfillset(sigset_t *ensemble);
int       sigismember(sigset_t *ensemble, int numero_signal);
int       sigpending(sigset_t *ensemble);
int       sigprocmask(int comment, const sigset_t *ensemble, sigset_t *ancien);
int       sigqueue(pid_t pid, int numero_signal, const union sigval valeur);
int       sigsuspend(const sigset_t *ensemble);
int       sigwaitinfo(const sigset_t *ensemble, siginfo_t *info);
int       sigtimedwait(const sigset_t *ensemble, siginfo_t *info,
                      const struct timespec *timeout);
unsigned int sleep(unsigned int nombre_secondes);
int       stat(const char *chemin, struct stat *tampon);

```

```

time_t      time(time_t *temps);
int         truncate(const char *chemin, off_t offset);
int         ftruncate(int fd, off_t offset);
int         truncate64(const char *chemin, off64_t offset);
int         ftruncate64(int fd, off64_t offset);
int         unlink(const char *nomFichier);
pid_t       wait(int *statut);
pid_t       waitpid(pid_t pid, int *statut, int options);
ssize_t     write(int fd, const void *tampon, size_t taille);

```

Annexes : constantes associées à des paramètres ou des champs de structures

```

options (open, fcntl) : O_RDONLY, O_WRONLY, O_RDWR, O_CREAT, O_EXCL, O_TRUNC, O_APPEND
mode (creat, open) : S_IRWXU, S_IRUSR, S_IWUSR, S_IXUSR, S_IRWXG, S_IRGRP, S_IWGRP, S_IXGRP, S_IRWXO, S_IROTH,
S_IWOTH, S_IXOTH
commande (fcntl) : F_GETFL, F_SETFL
mode (fopen, fdopen, freopen) : "r", "r+", "w", "w+", "a", "a+"
point_depart (lseek, fseek) : SEEK_SET, SEEK_CUR, SEEK_END
mode (mkfifo) : O_RDONLY, O_WRONLY, O_CREAT, O_EXCL, S_IRWXU, S_IRUSR, S_IWUSR, S_IXUSR, S_IRWXG, S_IRGRP,
S_IWGRP, S_IXGRP, S_IRWXO, S_IROTH, S_IWOTH, S_IXOTH
handler (signal, champ sa_handler de struct sigaction) : SIG_IGN, SIG_DFL
sa_flags (champ sa_flags de struct sigaction) : SA_ONESHOT, SA_NOMASK, SA_SIGINFO
comment (sigprocmask) : SIG_BLOCK, SIG_UNBLOCK, SIG_SET
options (waitpid) : WNOHANG, WUNTRACED

```

Annexes : macros

Sur `statut` de `wait`, `waitpid` : `WIFEXITED`, `WEXITSTATUS`, `WIFSIGNALED`, `WTERMSIG`, `WIFSTOPPED`, `WSTOPSIG`
 Sur le champ `st_mode` de `struct stat` : `S_ISREG`, `S_ISDIR`, `S_ISFIFO`

Annexes : quelques structures C

```

struct stat {
    dev_t      st_dev;
    ino_t      st_ino;
    mode_t     st_mode;
    nlink_t    st_nlink;
    uid_t      st_uid;
    gid_t      st_gid;
    dev_t      st_rdev;
    off_t      st_size;
    blksize_t  st_blksize;
    blkcnt_t   st_blocks;
    time_t     st_atime;
    time_t     st_mtime;
    time_t     st_ctime;
};

struct dirent {
    char d_name[256];
};

struct sigaction {
    void (*sa_handler) (int);
    void (*sa_sigaction) (int, siginfo_t*, void*);
    sigset_t sa_mask;
    int sa_flags;
};

union sigval_t {
    int sival_int;
    void *sival_ptr;
};

struct siginfo_t {
    int si_signo;
    int si_code;
    sigval_t si_value;
    pid_t si_pid;
    ...
};

struct timespec {
    long tv_sec;
    long tv_nsec;
};

```